

MUSTAFACODE

Lecture 09

Copy Constructor, Shallow Copy, Deep Copy, Destructor, this
Pointer & Accessor Functions (C++)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

Lecture 9: Copy Constructor, Shallow Copy, Deep Copy, Destructor, this Pointer & Accessor Functions (C++)

Chapter 09 — Copy Constructor, Shallow Copy, Deep Copy, Destructor, this Pointer & Accessor Functions (C++)

◆ 09.1 Copy Constructor

Definition

Copy constructor ek special constructor hota hai jo ek object ki values ko doosre object mein copy karta hai jab naya object create hota hai.

Kab use hota hai?

Copy constructor do situations mein automatically call hota hai:

1. Object initialization ke time

```
Student studentB = studentA;
```

2. Function ko object by value pass karne par

```
void func1(Student student){  
}
```

Copy Constructor Syntax

```
Student::Student(const Student &obj){  
    rollNo = obj.rollNo;  
    name = obj.name;  
    GPA = obj.GPA;  
}
```

Compiler Behavior

Agar hum copy constructor na likhein to compiler default copy constructor bana deta hai jo **shallow copy** karta hai.

◆ 09.2 Shallow Copy

Definition

Shallow copy mein sirf values copy hoti hain, lekin agar pointer ho to dono objects same memory share karte hain.

Example Problem (Shallow Copy)

```
Student studentA("AHMAD",1);  
Student studentB = studentA;
```

! Problem:

- `studentA.name` aur `studentB.name` same memory ko point karte hain
- agar ek object delete ho jaye to doosra dangling pointer ban jata hai

Dangling Pointer Problem

Agar `studentA` destroy ho jaye:

- memory free ho jati hai

- lekin `studentB.name` still usi memory ko point karta hai ❌

Real Life Example

Shallow copy = ek hi notebook share karna

→ ek banda erase kare to doosra bhi effect ho jata hai

◆ 09.3 Deep Copy

Definition

Deep copy mein har object ke liye separate memory allocate hoti hai.

Solution (Deep Copy Constructor)

```
Student::Student(const Student &obj){  
    int len = strlen(obj.name);  
  
    name = new char[len+1];  
    strcpy(name, obj.name);  
  
    rollNo = obj.rollNo;  
}
```

Benefit

- Har object independent hota hai
- koi dangling pointer issue nahi hota

Real Life Example

Deep copy = photocopy of notebook

→ har kisi ke paas apni copy hoti hai

◆ 09.4 Destructor

Definition

Destructor ek special function hota hai jo object destroy hone par automatically call hota hai.

Syntax

```
~Student(){  
    delete []name;  
}
```

Important Points

- Memory free karta hai (cleanup)
 - Automatically call hota hai
 - Overload nahi ho sakta
 - Sirf 1 destructor hota hai per class
-

Order of Execution

Constructors:

- ➔ creation order mein call hote hain

Destructors:

- ➔ reverse order mein call hote hain
-

Example Output

```
Ali Constructor  
Ahmad Constructor  
Ahmad Destructor  
Ali Destructor
```

◆ 09.5 Accessor Functions

Definition

Private data ko access karne ke liye functions use hote hain.

Types

1. Setter (Mutator)

Value set karne ke liye

```
void setRollNo(int aRollNo){  
    rollNo = aRollNo;  
}
```

2. Getter (Accessor)

Value return karne ke liye

```
int getRollNo(){  
    return rollNo;  
}
```

Validation Example

```
void setRollNo(int aRollNo){  
    if(aRollNo < 0){  
        rollNo = 0;  
    }  
    else{  
        rollNo = aRollNo;  
    }  
}
```

Good Practice

- Getter se direct reference return nahi karna chahiye
 - warna data modify ho sakta hai
-

◆ 09.6 this Pointer

Definition

`this` pointer current object ka address hold karta hai.

Important Concept

- har object ka data alag hota hai
 - functions shared hotay hain
 - `this` pointer batata hai kaunsa object use ho raha hai
-

Memory Concept

Objects:

s1 → data

s2 → data

s3 → data

Functions:

shared space (same for all)

Internal Working

```
void setRollNo(int aRollNo);
```

Actually compiler convert karta hai:

```
void setRollNo(int aRollNo, Student * const this);
```

Simple Explanation

- `this` = current object ka address
- compiler automatically pass karta hai

FINAL SUMMARY

✓ Copy Constructor

Object ko copy karta hai

✓ Shallow Copy

Same memory share hoti hai (dangerous with pointers)

✓ Deep Copy

Separate memory allocation (safe)

✓ Destructor

Memory free karta hai automatically

✓ Accessor Functions

Private data ko safe access deta hai

✓ this Pointer

Current object ko represent karta hai



EXAM TIP

- Agar pointer ho → Deep Copy use karo
- Destructor always add karo
- this pointer is implicit (hidden parameter)
- Getter/Setter use karo for encapsulation

